

## 1. High Level Description of Project

The initial plan for this project was to turn a laptop keyboard into a USB keyboard by implementing a keyswitch matrix scanner on the FPGA, using SPI to send scancodes to the Pi Pico, and using `tinyusb` to send keypress events to a computer. Our TA suggested we add more complexity, so we decided to increase scope by using an I2S amplifier to play audio samples on each keypress.

Since the iCESugar-pro has an SD card slot, we decided that we'd store the audio samples in an SD card connected to the FPGA. Implementing the SD card protocol in Verilog swallowed up so much time that we finished it *during finals week*. By the time we realized our blunder, it was too late to return to our original project scope (just the keyboard), so we had to set aside the keyboard and double down on getting audio playback from the SD card working.

**Our current project description is as follows:** The FPGA acts as an SPI master for two devices: the SD card and the Raspberry Pi Pico. The FPGA initializes the SD card, then starts reading blocks and streaming them to the Pico. The Pico saves the raw audio data into memory before disabling the FPGA (via a pin separate to the SPI bus) and playing the audio sample in a loop.

## 2. Elements of Complexity

- SD card, driven and read by the FPGA
- I2S between Pico and amplifier
- SPI communication between FPGA and Pico
- Testbenches for Verilog code

## 3. User Guide

TODO: Instructions for flashing audio data onto an SD card

## 4. List of Hardware Components

- iCESugar-pro
- Raspberry Pi Pico
- Micro SD card
- MAX98357 I2S amplifier
- Generic speaker

## 5. List of Software Libraries

On the Raspberry Pi Pico 2, we used CircuitPython to process raw pcm data (place into a buffer) and output it as I2S.

On the FPGA, all the code was written by us.

## 6. List of Protocols Used

- SD card SPI protocol.
  - This is a protocol built on top of SPI. It uses fixed length, 6 byte commands. Most command responses are 1 byte, but some append extra data. We implemented it in Verilog.
- I2S

- This is a protocol that is technically handled by Circuit Python. It just outputs raw PCM data, a LR signal (for left and right stereo), and CLK at 512 kHz to the MAX98357A.

## 7. How we met the Requirements listed in the Proposal

In our project proposal? We don't meet any of the requirements at all. Our project scope has changed so much that our project proposal is not an accurate picture of what we delivered.

Ultimately though, we were able to successfully read data from the SD card.

We were also able to output this data onto a speaker, but the library that we used to output the I2S has some very strange implementations such that it only is able to load one sound and play it once or keep looping it. We understand the protocol completely, so if we had more time (it took 3 weeks to switch to using the library because the I2S implementation didn't work and the clock was off by 15 kHz which led to noise I think) then maybe we could have implemented something that had the capacity to play any data that was input into it rather than load data once.

We did not implement the keyboard nor the

## 8. Design Diagrams

### 8.1. Wiring

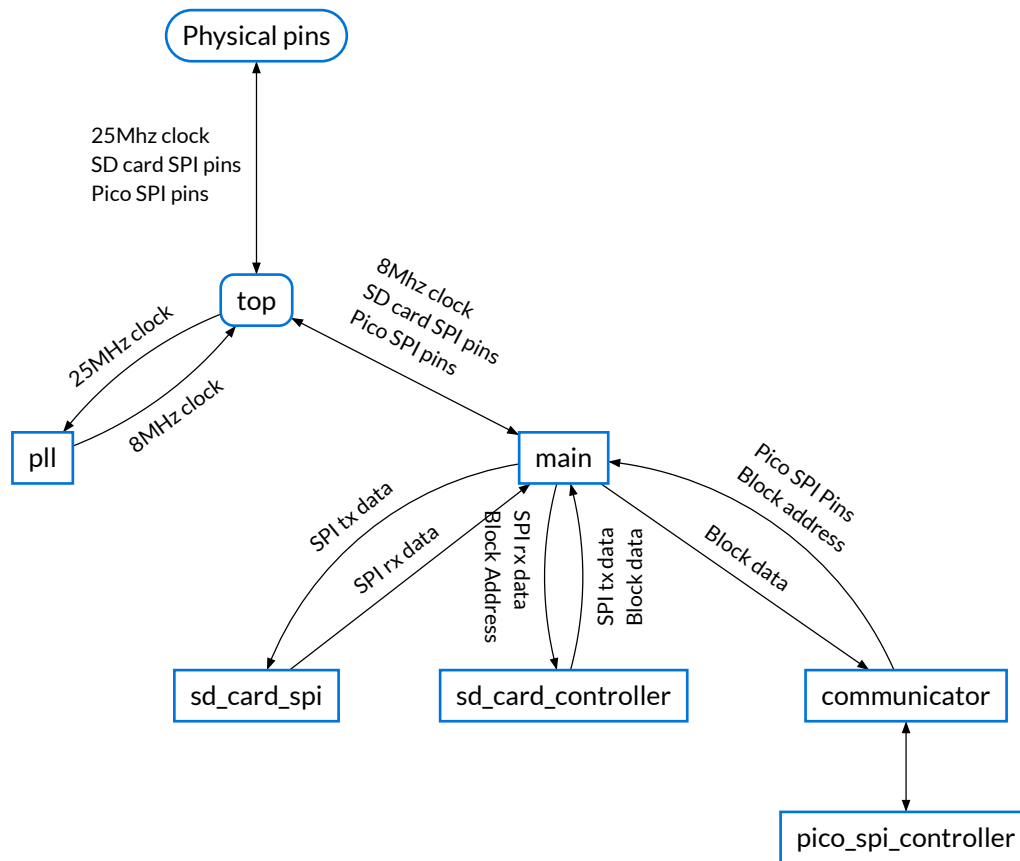
TODO: Maybe sketch something out on paper and scan it

### 8.2. SD Card Module State Machine

The Verilog module that handles communication with the SD card contains a large state machine.

TODO: Make a diagram of that state machine

### 8.3. FPGA Module Hierarchy



## 9. AI Usage

Neither team member used AI for any part of this project.

## 10. Acknowledgements

Massive thank you to RB for being a chill TA. Another thanks to Allan Knight for helping with debugging I2S on the fpga.

No other outside code was used for the Raspberry Pi Pico 2 or the FPGA except for the pll clock.